

# 캐치플라워 (CatchFlower) — AI 활용 기술 문서

AI를 두 층에서 썼다. 만드는 데 쓴 AI와 제품 안에서 도는 AI다. 이 문서는 그 둘을 갈라서, 어떤 도구에 무엇을 시켰고 결과를 어떻게 검증했는지를 적는다.

|            |                                                               |
|------------|---------------------------------------------------------------|
| 개발 도구      | Claude Code (Anthropic Claude Opus) — 터미널 2개 병행               |
| 제품 내 AI ①  | PlantNet API k-eastern-asia (4,932종) — 종 판별                   |
| 제품 내 AI ②  | ML Kit Image Labeling (온디바이스 · 번들 모델) — 꽃 여부 1차 필터            |
| 검토했다 안 쓴 것 | Gemini 2.5 Flash-Lite · Kindwise Plant.id · GPT-4 계열 (5절에 이유) |
| AI가 안 만든 것 | 꽃 일러스트 200장 (Figma · 사람 작업) · UI 문구 전체                        |
| 개발 기간      | 2026-08-04 ~ 08-10 (7일) · 진행 기록 (49)회차 · 커밋 46개               |
| 산출 규모      | Android main 14,789줄 + 테스트 10,084줄 · iOS 7,416줄 · 문서 11,377줄  |

## 1. 개발에 쓴 AI — Claude Code

### 1-1. 어떻게 돌렸나 — 터미널 2개를 계약서로 묶었다

iOS와 Android를 동시에 만들었다. AI 세션 두 개가 같은 서버·같은 DB를 건드리므로 한쪽이 필드명을 바꾸면 다른 쪽이 조용히 깨진다. 그래서 순서를 뒤집었다.

```
프로젝트 맥락/공유계약_iOS_AOS.md ← 두 세션이 혼자 바꿀 수 없는 것
├ DB 컬럼명 · enum 문자열
├ 정책 숫자 (임계값 · 반경 100m · 하루 1회 · 최소 인원 10명)
└ Mock 인식기의 동작 규칙
  ↓ 이 문서를 고치고 진행.md에 남긴 뒤에만 코드를 고친다
터미널 A [iOS · SwiftUI]      터미널 B [Android · Compose]
```

| 파일                        | 역할                 | 왜 필요했나                                                             |
|---------------------------|--------------------|--------------------------------------------------------------------|
| 공유계약_iOS_A0S.md           | 플랫폼이 혼자 못 바꾸는 것    | 한쪽이 ai_confidence 를 Double로 넓혀 다른 쪽 파싱이 깨진 일이 실제로 있었다 (진행.md (24)) |
| 세션시작_iOS.md / 세션시작_A0S.md | 새 세션이 가장 먼저 읽는 문서  | 컨텍스트가 끊겨도 환경·금지사항이 유지된다                                            |
| 구현현황_iOS.md / 구현현황_A0S.md | 지금 무엇이 되고 무엇이 안 되나 | "다 됐다"는 보고가 축적되는 것을 막는다                                            |
| 진행.md (4,770줄)            | 회차별 기록             | 무엇을 했는지가 아니라 무엇이 틀렸는지를 남긴다                                         |

인계 문서가 가장 위험했다. 세션시작\_A0S.md 가 나흘 낚아 "코드 6,870줄 · PlantNetRecognizer 가 0줄"이라고 말하고 있었다. 실제로는 14,789줄이고 실기기 실호출까지 됐다. 한때 맞았던 글은 거짓이 된 뒤에 결함을 가리는 쪽으로 작동한다 — 그래서 (49)회차를 이 문서 갱신에 썼다.

## 1-2. 실제로 쓴 프롬프트 — 설계를 바꾼 것만 뽑았다

전부 옮기면 (49)회차 분량이라, 결과물이 실제로 달라진 프롬프트만 적는다. 공통점은 짧고, 답을 지정하지 않고, 근거를 되묻는다는 것이다.

| # | 실제 프롬프트                                                   | AI가 하려던 것                 | 바뀐 결과                                                                                                |
|---|-----------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------|
| 1 | 1,000유로면 얼마나... 제미나이 api로 풀어보는 방법은 없나?                    | 전용 꽃 인식 API만 비교           | 범용 멀티모달 단가를 대보니 <b>자릿수가 달랐다</b> . 연 226만원 → 20만원 안이 나왔고 최종 설계가 바뀌었다                                  |
| 2 | 하루 500건 무료인데 연 20만 건이 1,000유로라는 거야?                       | "무료 500건/일" 표기를 물량으로 읽음   | 약관을 다시 읽으니 <b>물량이 아니라 라이선스 최소 계약 (€1,000)</b> 이었다. 비용 문서를 정정했다                                       |
| 3 | 정확도는 각각 어느정도이지?                                           | 벤더 표기(≈80%)를 근거로 채택       | 조사 결론이 " <b>공개 자료로는 결정할 수 없다</b> "였고, 그래서 <b>실측 200장</b> 을 설계했다                                      |
| 4 | 지금 너 너무 사진인식안되는거에 꽃힌거는 아니냐? 일단 서비스 작동을 위해서 필요한거 먼저 진행해도된다 | 정확도 튜닝을 계속 파고들            | 맞는 지적이었다. <b>DB가 0테이블</b> 이라 로그인·저장·랭킹이 시작조차 못 하는 상태였다. 스키마로 방향을 틀었다                                 |
| 5 | aos ios 개발 둘 다 할 것                                        | "iOS 먼저" 권고               | 두 플랫폼 동시 개발로 뒤집혔고, 그 결과가 <b>공유계약 문서</b> 다                                                            |
| 6 | 모든 아이콘을 적용시킬필요는 없다. 현재 디자인에 있는 아이콘들에 필요한 부분들을 적용시키는 거야    | 납품 아이콘 44종을 전부 넣으려 함      | <b>7종만 넣었다</b> . 안 쓰는 아이콘을 넣으면 그것을 놓을 자리를 만들게 되고, 그 자리는 <b>눌러도 아무 일 없는 버튼</b> 이 된다                   |
| 7 | 화면상에 있는 사진을 찍어도 허용되게 잠시 열어둘수 있나?                          | (막힌 이유를 결함으로 오해할 수 있던 지점) | 모니터 속 꽃은 모아레 때문에 ML Kit이 <b>Pattern</b> 으로 읽는다 — <b>필터가 맞았다</b> . QA용 파일 스위치를 따로 만들어 뒤쪽 흐름을 손으로 검증했다 |
| 8 | 전체 권한줄게 쪽쪽진행해 api키 필요한 경우그런경우만 답변 요청하고                    | 매 단계 확인을 요청               | <b>막히는 지점을 "키가 필요한 경우"로 한정</b> 했다. 그 외에는 판단하고 기록으로 남기는 방식으로 바꿨다                                      |

두 번 연속으로 "이게 얼마나"는 되물기가 설계를 바꿨다(#1·#2). AI가 만든 비교표는 정교했지만 후보 집합 자체가 좁았다. 가격표를 정밀하게 비교해도 후보가 빠져 있으면 정밀함이 무의미하다 — 이 프로젝트에서 AI 사용법의 핵심 교훈이다.

### 1-3. AI에게 준 상시 규칙 — 프롬프트보다 이게 더 많이 일했다

매번 말하는 대신 문서로 고정해 세션마다 읽히게 했다.

| 규칙                       | 왜                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------|
| UI 문구를 새로 쓰지 않는다         | 전부 디자이너_업무/A_문구_버튼_스펙.md 에 있다. 없으면 A 문서에 먼저 추가하고 기록한 뒤 쓴다                         |
| 생성물을 직접 고치지 않는다          | CSV·SVG·xlsx·SQL은 _tools/*.py 가 원본이다. 결과 파일을 고치면 다음 생성에서 날아간다                     |
| 키를 소스에 넣지 않는다            | local.properties → buildConfigField → AppSecrets . 검증은 이름과 길이로만 하고 값을 절대 출력하지 않는다 |
| 실측 보고 전에 유료 API를 부르지 않는다 | PlantNet 누적 호출을 세어 기록한다 (현재 16건 / 500건·일)                                         |
| 원본 응답을 캐시한다              | 채점 규칙을 고쳐도 쿼터를 다시 쓰지 않는다. 실제로 floor 재채점을 4번 했다                                    |
| 계약값을 혼자 바꾸지 않는다          | 필드명·enum·정책 숫자는 공유계약을 고치고 기록한 뒤에만                                                 |
| 회차마다 진행.md에 남긴다          | 컨텍스트가 끊겨도 결정의 근거가 남는다                                                             |

#### 1-4. AI가 만든 것을 어떻게 믿지 않았나

이 프로젝트에서 가장 많은 시간을 쓴 부분이다. AI는 "됐습니다"를 아주 잘 말한다.

|        |                                  |
|--------|----------------------------------|
| 컴파일 성공 | → 증거가 아니다 (진행.md (17))           |
| 테스트 초록 | → 증거가 아니다 (진행.md (22)(31)(32))   |
| 정책이 붙음 | → 규칙이 도는 것과 다르다 (진행.md (23))     |
| 빌드 성공  | → 검사 대상이 안 읽혔을 수 있다 (진행.md (41)) |

| 실제로 겪은 것                                            | 증상               |
|-----------------------------------------------------|------------------|
| 테스트 249개가 전부 초록인 상태에서 앱이 죽고 화면에 null 이 떠 있었다        | 초록               |
| 랭킹이 정렬이 틀린 채로 예쁘게 렌더됐다                              | 정상처럼 보임          |
| answered > 0 이라는 느슨한 단정 때문에 점수 0.002 하나로 통과했다       | 초록               |
| res/ ·문서를 읽는 검사 12개가 Gradle 입력에 없어서 안 돌면서 초록이었다     | BUILD SUCCESSFUL |
| RLS 정책이 대시보드에 다 붙어 있는데 정책 안의 서브쿼리가 다시 RLS를 타서 구멍 3곳 | 초록               |

해법으로 쓴 것: 돌연변이 검증. 검사를 믿기 전에 코드를 일부러 틀리게 바꿔 그 검사가 실제로 빨개지는지 확인한다. 누적 182건을 이렇게 확인했다.

| 돌연변이 | 되돌린 것                   | 결과                                   |
|------|-------------------------|--------------------------------------|
| 179  | floor 0.30 → 0.05       | expected:<77> but was:<42> 로 빨감      |
| 180  | 0.05를 0.30으로 되돌림        | 화면 12로 가는 비율을 기록한다 FAILED            |
| 181  | preFilter 를 항상 통과하도록 교체 | FAILED (QA 스위치가 켜진 채 배포되는 것을 막는다)    |
| 182  | 진단 로그에서 학명을 제거          | 응답 요약 로그가... FAILED → 복원 후 바이트 동일 확인 |

살아남은 돌연변이도 있었고 그게 더 값졌다. RGB\_565 → ARGB\_8888 을 되돌린 돌연변이는 값 테스트 4개를 전부 통과했다 — 픽셀도 계산도 안 틀리고 투명 배경만 검게 칠해진다. 검사가 볼 수 없는 층이 있다는 뜻이고, 그건 문서에 적었다.

### 1-5. 이 제출물도 스크립트다

이 PDF를 손으로 만들지 않았다. 제출물/\_tools/build\_pdf.py 가 마크다운을 변환한다 (Chrome headless 인쇄 — 이 맥에 pandoc이 없고 brew install --cask 가 막혀 있다). PDF를 손으로 고치면 원본과 어긋나기 때문이고, 그건 이 프로젝트의 생성물 규칙과 같다.

변환기에도 테스트를 붙였다( test\_build\_pdf.py ). 이유는 조용히 틀리는 방식이 있어서다 — 표 칸 안의 ` | ` 가 칸을 쪼개고, 2 \* 3 \* 4 가 2 <em> 3 </em> 4`가 되고, 이미지 경로가 틀려도 Chrome은 PDF를 정상 생성하고 그 칸만 빈다 (오류도 안 난다). 셋 다 실제로 걸렸고, 마지막 것은 PDF를 열어 봐도 눈에 안 띈다.

## 2. 제품 안에서 도는 AI ① — PlantNet (종 판별)

### 2-1. 무엇을 어떻게 부르나

|       |                                                                                    |
|-------|------------------------------------------------------------------------------------|
| 엔드포인트 | https://my-api.plantnet.org/v2/identify/k-eastern-asia                             |
| 모델 범위 | 4,932종 (동아시아 프로젝트)                                                                 |
| 요청    | multipart/form-data · organs=flower · nb-results=10 · include-related-images=false |
| 사진    | 긴 변 1,600px로 리사이즈해서 보낸다 (원본을 그대로 보내지 않는다)                                          |
| 응답    | 학명 + 점수 배열                                                                         |
| 요금    | 무료 등급 일 500건 · 현재 누적 실패출 16건                                                       |

organs=flower 를 명시한다 — 우리가 받는 사진은 항상 꽃이라 기관을 알려주면 정확도가 오른다.

## 2-2. AI 답을 그대로 쓰지 않는다 — 세 겹을 통과해야 후보가 된다

사진

- ↳ ① ML Kit 온디바이스 필터      꽃이 아니면 여기서 끝 (유료 호출 안 함)
- ↳ ② 개화월 하드 필터              지금 피는 종만 후보 (8월 = 98종)
- ↳ ③ PlantNet 실호출
- ↳ ④ 학명 색인                      도감 200종에 없는 학명은 버린다
- ↳ ⑤ 후보 3개 + floor 0.05
- ↳ 화면 09 (사용자가 고른다) / 화면 12 (실패)

② 개화월 하드 필터는 선택이 아니라 필수다. PlantNet은 4,932종 중에서 고르므로 11월에 "벚꽃"을 1순위로 준다. 그런데 화면에는 그냥 "AI가 좀 틀렸네"로 보인다.

⑤ 최종 결정은 AI가 아니라 사람이 한다. 점수가 높아도 이 꽃이에요 / 다른 꽃이에요 를 묻고, 애매하면 후보 3개를 나란히 놓는다. 유사종(민들레 vs 서양민들레)은 사람도 헷갈리므로 AI가 단정하지 않게 설계했다.

## 2-3. 실측 — 200장을 라벨로 재고, 원본 응답을 전부 캐시했다

TF-flowers 5클래스 × 40장. `ios/Tools/plantnet_measure.py` 가 앱과 같은 규칙을 재현한다(개화월 필터 + 학명 색인 + 후보 3개 + floor). 응답 200개를 캐시했으므로 채점 규칙을 고쳐도 쿼터를 다시 쓰지 않는다.

| 클래스        | 원시 Top-1 | 앱 Top-1 |
|------------|----------|---------|
| sunflowers | 95.0%    | 95.0%   |
| tulips     | 95.0%    | 95.0%   |
| dandelion  | 72.5%    | 90.0%   |
| roses      | 62.5%    | 62.5%   |
| daisy      | 55.0%    | 42.5%   |
| 합계         | 76.0%    | 77.0%   |

⚠ 이 77%를 "합격선 통과"로 쓰지 않았다. 표본이 쉽다 — 도감의 AI 난이도 '상' 51종은 여기 없다. 실측 보고를 한 번 정정한 기록이 `진행.md` (21)에 있다. 지표가 쉬운 표본에서 나오면 합격이 아니라 상한이다.

## 2-4. 실측이 잡은 것 — 정답 0.606이 오답 0.003에게 지고 있었다

원시 76%인데 앱은 21%였다. 60%p 차이의 원인은 API가 아니라 우리 코드였다.

0.606 *Rosa chinensis* → 속 대표를 도감번호 최솟값으로 고정 → 찔레꽃(5~6월) → 8월 탈락  
0.003 *Begonia grandis* → 베고니아 → 1순위로 채택

도감 200종 중 94종이 다종 속에 있고, 그중 30속은 개화월이 서로 다르다. 예외가 아니라 절반의 문제였다. 색인이 호출 시점의 후보 집합을 받아 속 안에서 제철 종을 고르게 고쳤다.

왜 앱을 커서는 못 봤나. 8월에 장미를 찍으면 베고니아 가 뜨는데 그게 그냥 오인식으로 보인다. 달을 바꿔 재야 드러난다 — 한 달로 고정된 측정은 원인을 귀속할 수 없다.

2-5. AI 점수를 신뢰도로 착각하지 않았다 — floor 0.30 → 0.05

PlantNet 점수는 4,932종에 퍼진 확률이라 정답도 0.1 근처에 흔히 온다 (정답 사진의 25% 분위가 0.157). 처음 실패 임계값을 0.30으로 뒀더니 버려진 4장이 전부 정답이었다(실기기 실측 — 장미 0.110 · 서양민들레 0.112 · 0.185 · 0.019).

| floor             | 1순위 정답률 |
|-------------------|---------|
| 0.30 (처음)         | 50%     |
| 0.10              | 76%     |
| 0.05 (현재 · 오너 승인) | 80%     |

낮춰도 오등률이 늘지 않는 이유는, 이 값이 "확정" 관문이 아니라 "후보로 보여줄 자격" 이기 때문이다. 점수가 낮으면 화면 09에서 사용자가 고른다. 0.05는 iOS·Android 공용 계약값이라 어느 한쪽이 혼자 바꿀 수 없다.

3. 제품 안에서 도는 AI ② — ML Kit 1차 필터 (온디바이스)

유료 호출을 줄이려고 무료·온디바이스 AI를 앞에 뒀다. 사진 1,000장으로 실측했다 (TF flowers 500 + Imagenette 500 · Pixel 8 에뮬레이터 API 36).

| 항목            | 실측                                 |
|---------------|------------------------------------|
| 재현율 (꽃을 통과시킴) | 100.0% — 막힌 정상 촬영 0장               |
| 차단율 (꽃아님을 막음) | 92.4%                              |
| 지연            | 중앙값 19~20ms (p90 22ms)             |
| APK 증가        | arm64 기준 약 14MB (so 11MB + 모델 3MB) |
| 과금            | 0원 (번들 모델 · 네트워크 없음)               |

확정 설정: 라벨 Flower · Petal · Blossom / 임계값 0.20.

처음엔 Plant · Leaf · Garden 도 넣어 뒀다. "근접 촬영이면 Plant가 더 높게 나올 것"이라는 추측이었는데 측정은 정반대였다 — Plant 는 꽃아님 사진 489/500장에도 붙는다(배경 초목). 넣으면 차단율이 96.6% → 62.8%로 무너지면서 재현율은 오르지 않는다.

● 가장 위험했던 함정: 데이터셋 라벨 노이즈가 잘못된 설정을 유도한다. 처음 재현율이 96%로 나왔다. "놓친 꽃 20장"을 전부 열어봤더니 16장이 꽃이 아니었다 — 행진 악대, Roses 상점 간판, 매달린 찻잔, 금속 풍선, 벽돌 건물. ML Kit이 맞고 데이터셋이 틀렸다. 이걸 모르고 96%를 보면 임계값을 낮추게 되고, 그러면 차단율이 무너져 비용 절감 자체가 사라진다. 숫자만 보고는 절대 못 잡는다.

임계값을 0.30이 아니라 0.20으로 내린 근거 (DAU 1,000 환산):

| 임계값       | 허출     | 막힌 정상 촬영 |
|-----------|--------|----------|
| 0.20 (채택) | 476건/월 | 0건       |
| 0.30      | 213건/월 | 201건     |

263건 아끼려고 손에 진짜 꽃을 들고 있는 201명에게 꽃이 아닐 수도 있어요 를 보여주는 거래다. 0.30 이상에서 막히는 사진을 전부 열어보니 역광 민들레 흙사·흑백 접사· Flower 0.28 로 간신히 못 넘긴 근접 촬영 — 우리 사용자가 실제로 찍은 사진들이었다.

측정 코드는 출력만 하지 않고 단정한다(재현율  $\geq 99\%$  · 차단율  $\geq 90\%$  · 지연  $\leq 200\text{ms}$ ). 출력만 하면 나중에 라벨셋을 건드려도 테스트가 계속 통과한다.

## 4. AI 실패를 화면이 가리지 않게 했다

어떤 꽃인지 알 수 없었어요 (화면 12)는 원인이 셋인데 화면에는 똑같이 보인다.

|   | 원인                         | 우리 책임인가         |
|---|----------------------------|-----------------|
| Ⓐ | PlantNet이 아무것도 못 알아봤다      | 아니다 (API 한계)    |
| Ⓑ | 알아봤지만 도감 200종·개화월에서 전부 걸렸다 | 우리 도감이 좁다 (B-4) |
| Ⓒ | 통과했지만 1순위가 floor 미달이다      | 우리 정책이다 (B-3-a) |

로그 네 줄로 갈라 두고, 그 로그의 내용까지 테스트로 고정했다 — 로그는 조용해져도 증상이 없기 때문이다.

```
1차필터: 꽃=true top=Flower(0.71)
판별 호출 직전: PlantNetRecognizer 후보=98개 (8월) ← 여기가 과금 시점
판별 응답: 원시 7종 → 학명 매칭 2종 → floor 통과 1종
화면 12: 원인=Ⓒ (1순위 0.031 < floor 0.05)
```

이 구분이 없어서 "API가 문제인 줄 알았는데 우리 임계값이었다"를 한 번 겪었다. 그리고 판별 호출 직전 줄이 있어야 어느 촬영이 과금됐는지 사후에 확인할 수 있다 — "몇 건 썼는지 모른다"는 상태를 만들지 않았다.

## 5. 검토하고 안 쓴 AI — 왜 버렸는지가 근거다

### Gemini 2.5 Flash-Lite — 가장 싸지만 점수를 못 믿는다

단가는 압도적이었다(건당 0.68원 · DAU 1,000에서 연 20만원 vs PlantNet Pro 226만원). 그런데 버렸다.

|        | PlantNet (분류 모델)      | Gemini (LLM)       |
|--------|-----------------------|--------------------|
| 점수의 의미 | 보정된 확률 — 0.72가 실제 72% | "95% 확신합니다"라고 말할 뿐 |
| 모를 때   | 낮은 점수를 준다             | 그렇듯하게 지어낸다         |

우리 판별 설계(난이도별 임계값 60/70/85 + floor)는 보정된 확률을 전제로 만들어졌다. 그리고 난이도 '상' 51종이 진짜 시험대다 — 민들레 vs 서양민들레처럼 사람도 헷갈리는 데서 LLM은 그럴듯한 답을 자신 있게 낸다. 오등록은 "이 앱 틀렸다"로 오히므로 정확도보다 위험하다.

덧붙여 Gemini 무료 등급은 데이터를 제품 개선에 사용한다. 사용자 사진 + 위치를 보내는 앱이라 무료 등급을 쓸 수 없다.

버리지 않고 남긴 자리: FlowerRecognizer 인터페이스 뒤의 대안 엔진, 그리고 꽃 이야기 200종 초안 생성(아직 미사용 — 쓰게 되면 식물 정보라서 출처 표기가 필수다).

### Kindwise Plant.id — 연 135만~677만원. 무료 등급이 없다

### GPT-4 계열 — 비교 자료가 우리 조건과 안 맞았다

찾은 유일한 비교(Kindwise 블로그 · 50장 · 2024-02 · GPT-4 Turbo 42%)는 ① 벤더가 자기 제품과 비교한 것이고 ② 60%가 실내 관엽식물(우리는 한국 야생화)이고 ③ 2024-02 모델이라 2세대 전이다. 42%는 현재 추정치 아니라 하한선이다.

"자료가 없다"를 확인한 것도 성과였다. 공개 자료로 결정할 수 없다는 결론이 실측 200장 설계를 낳았고, 그 실측이 우리 코드 버그(2-4절)를 잡았다.

## 6. AI가 만들지 않은 것

정직하게 갈라 둔다. AI 활용 문서에서 이 절이 빠지면 나머지 절도 못 믿는다.

| 항목             | 만든 방법                                                                                      |
|----------------|--------------------------------------------------------------------------------------------|
| 꽃 일러스트<br>200장 | 사람 작업 (Figma) — PNG 512×512 RGBA. 생성 AI 아님. 발주서는 <code>디자이너_업무/C_꽃일러스트_발주서.md</code>       |
| 앱·인앱 아이콘       | 사람 작업 (납품 44종 중 7종만 적용)                                                                    |
| UI 문구 전체       | 사람이 확정한 <code>A_문구·버튼_스펙.md</code> 가 원본. 코드에서 문구를 창작하지 않는다 (금지어 목록이 아니라 승인된 문구만 허용으로 뒤집었다) |
| 도감 200종 데이터    | 사람이 정한 목록 + <code>꽃도감/_tools/flowers.py</code> 가 생성·검증 (개화월 파싱은 한 곳에서만)                    |
| 정책 숫자 35항목     | 오너 결정 ( <code>오너_결정사항.md</code> 가 원본) — floor 0.05를 포함해 AI가 혼자 정한 값이 없다                    |

## 7. AI 활용 요약 — 무엇이 실제로 효과가 있었나

| 한 일                     | 효과                                                                   |
|-------------------------|----------------------------------------------------------------------|
| AI를 두 세션으로 나누고 계약서로 묶었다 | iOS·Android를 7일에 동시 개발. 필드명 충돌은 계약 문서에서 잡혔다                          |
| 회차마다 "무엇이 틀렸는지"를 기록     | 4,770줄 · (49)회차. 컨텍스트가 끊겨도 결정의 근거가 남는다. 같은 함정을 두 번 밟은 경우까<br>지 적혀 있다 |
| 되문기로 후보 집합을 넓혔다         | "이게 얼마나" 두 번이 비용 설계를 두 번 뒤집었다 (226만원 → 0원)                           |
| 초록불을 증거로 쓰지 않았다         | 돌연변이 182건. 초록 249개 상태에서 앱이 죽어 있던 경험이 이 습관을 만들었다                      |
| AI 답을 사람이 최종 결정         | 화면 09에서 사용자가 고른다. 오등록이 정확도보다 위험하다는 판단                                |
| AI 실패의 원인을 세 갈래로 로깅     | "API가 문제인 줄 알았는데 우리 정책이었다"를 다시 겪지 않는다                                |
| 원본 응답 캐시                | floor를 4번 재채점하면서 유료 호출 0건 추가                                         |

가장 크게 배운 것 하나만 남긴다. AI는 요구한 것을 잘 만들고 "됐습니다"를 아주 잘 말한다. 그래서 이 프로젝트에서 가장 많은 시간을 쓴 일은 기능 추가가 아니라 "이 검사가 어떤 경우에 빨개지는가"를 먼저 답하는 것이었다. 컴파일 성공 · 테스트 초록 · 정책 불음 · 빌드 성공 — 네 가지 다 성공처럼 생긴 출력을 낸다.

## 8. 함께 보는 문서

| 문서                                                        | 내용                                      |
|-----------------------------------------------------------|-----------------------------------------|
| 제출물/게임_소개.md                                              | 게임 개요 · 플레이 방법 · 실행 방법                  |
| 프로젝트 맥락/진행.md                                             | <b>개발 전 과정 (49)회차</b> — 이 문서의 모든 숫자의 출처 |
| 프로젝트 맥락/공유계약_iOS_A0S.md                                   | 두 AI 세션이 혼자 못 바꾸는 것                     |
| 프로젝트 맥락/AI인식_API_비용검토.md                                  | 5절 비용·정확도 비교의 원본 (세 번 정정한 이력 포함)        |
| 프로젝트 맥락/오너_결정사항.md                                        | 사람이 정한 35항목                             |
| android/app/src/main/java/com/catchflower/app/recognizer/ | 판별 코드 — 필터 · 색인 · floor · 로그            |